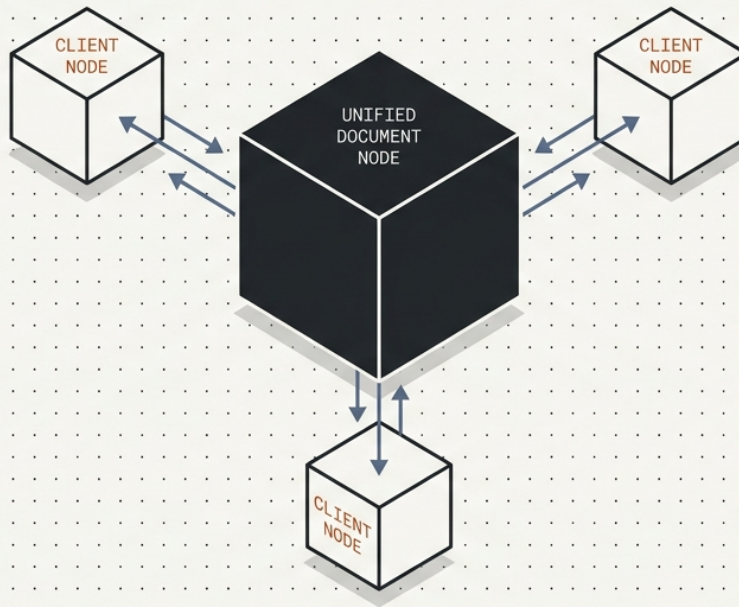


Engineering real-time collaborative editing at global scale

EST. INGEST: 18Gbps

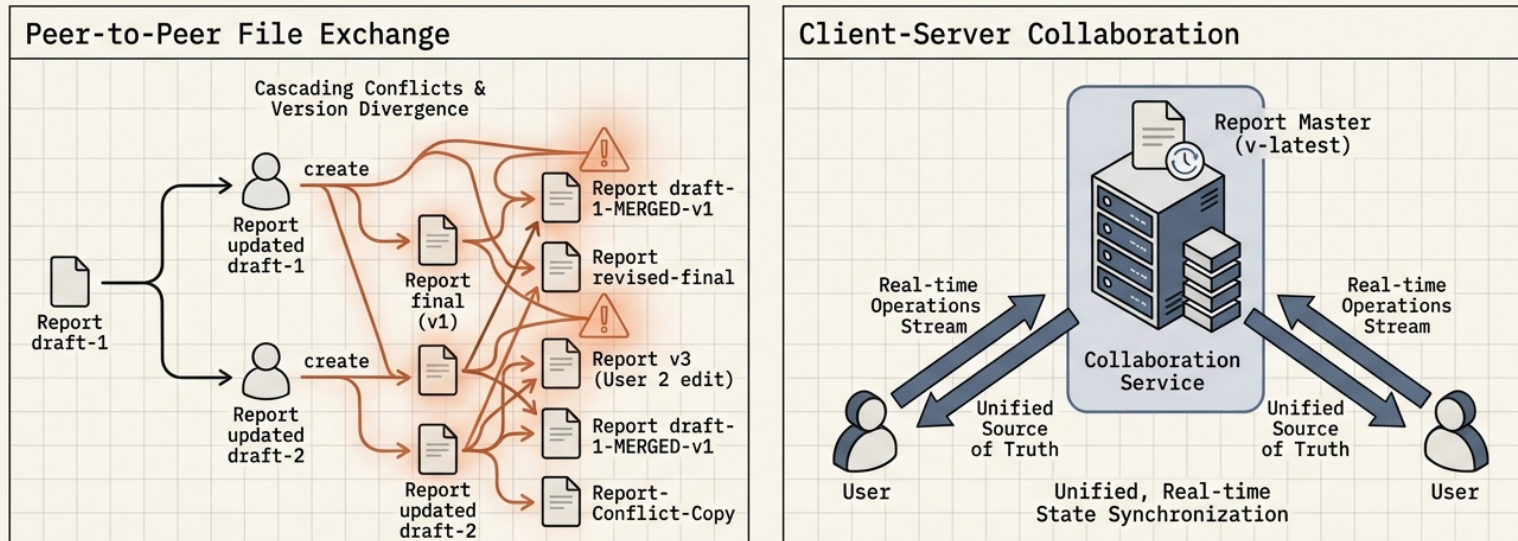
DAU: 80,000,000



LATENCY: <50ms

STATE: CONVERGED

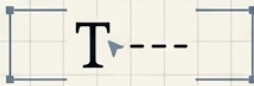
Unsynchronized local editing creates cascading version conflicts



Manual file exchange breaks commutativity and creates friction. Granular state control requires transitioning to a centralized client-server architecture.

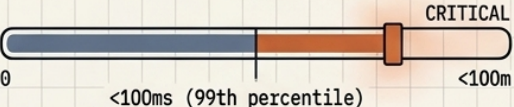
DEFINING THE FUNCTIONAL AND NON-FUNCTIONAL SYSTEM CONSTRAINTS

FUNCTIONAL REQUIREMENTS

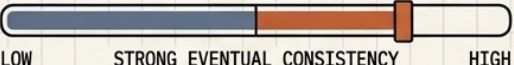
	Real-time stream of operations
	Conflict resolution logic
	Autocomplete suggestions
	Collaboration history
	Document view counts

NON-FUNCTIONAL CONSTRAINTS

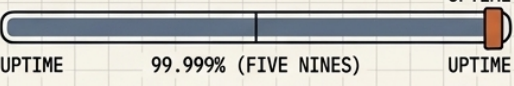
Latency: Must support real-time cross-region updates



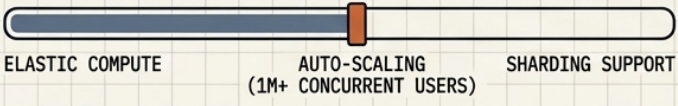
Consistency: Must mathematically resolve concurrent overwrites



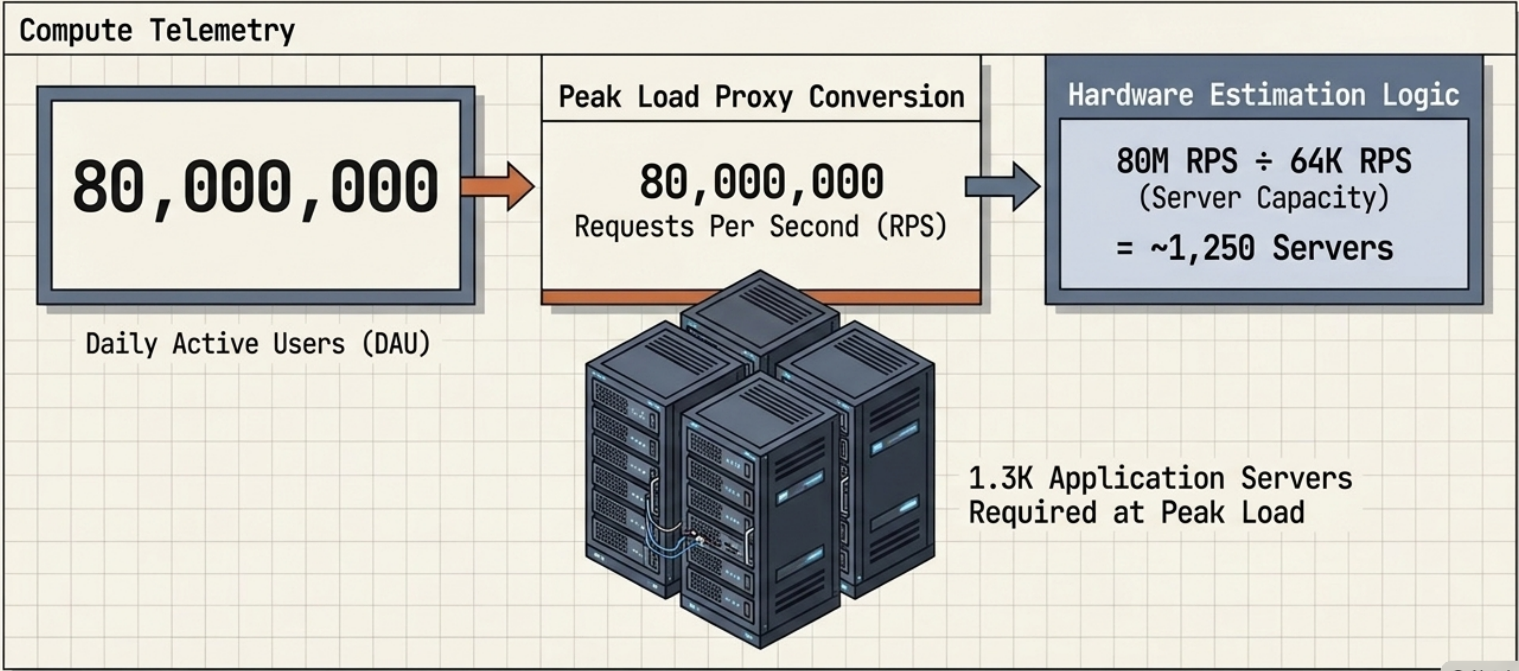
Availability: Continuous uptime required



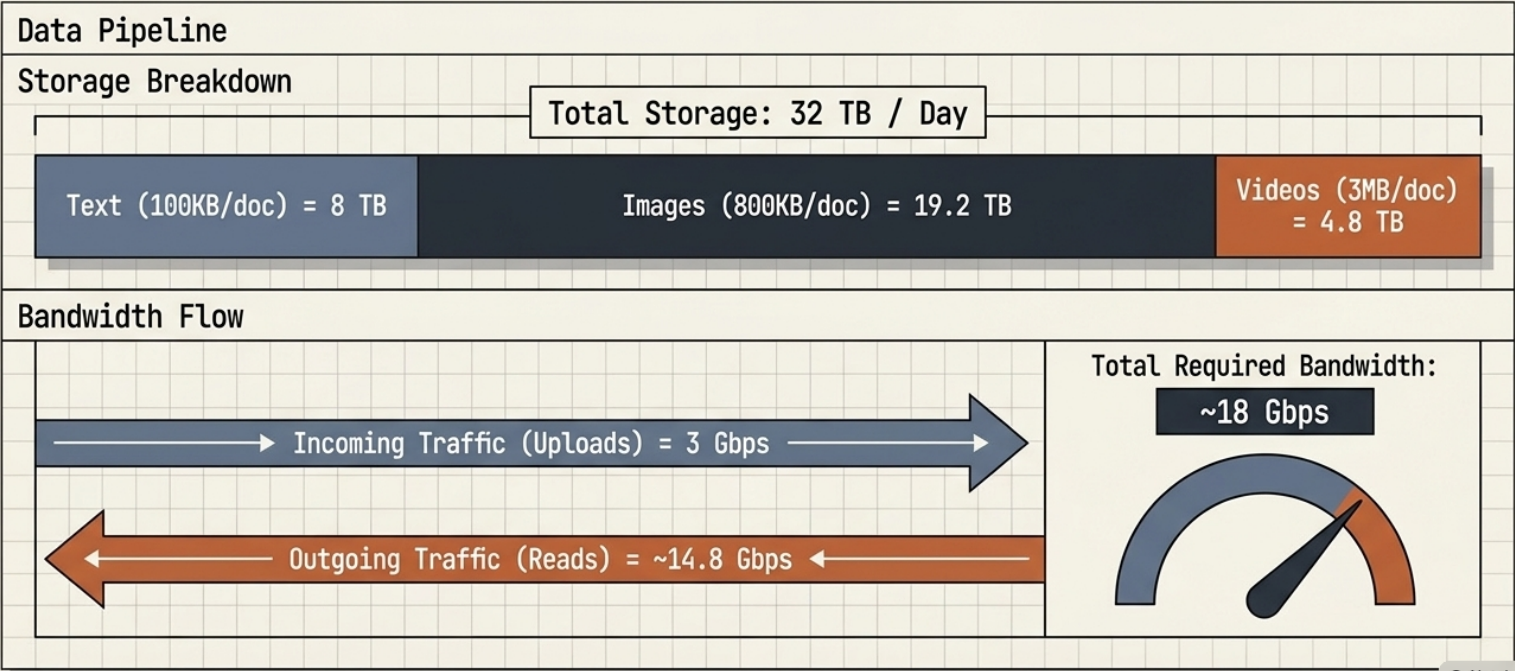
Scalability: Massive concurrent reader/writer loads



80 million daily active users require a 1.3K application server cluster



Daily media and text ingestion demands a 32 TB storage and 18 Gbps pipeline



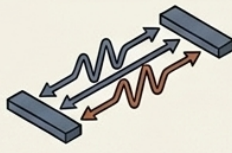
Mapping the distributed building blocks required for collaborative editing

API Gateway



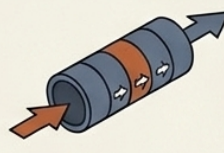
Intercepts/routes client requests and handles auth.

WebSockets



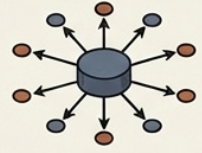
Persistent transport layer continuous character edits.

Processing Queue



Batches frequent, small character insertions.

Pub-Sub (Kafka)



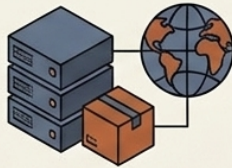
Manages async tasks like view counts.

Time-Series DB



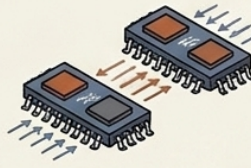
Preserves exact operational order for document history.

Blob Storage & CDN



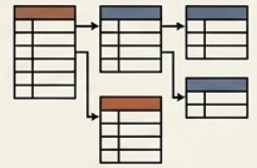
Caches heavy static media natively.

NoSQL & Redis



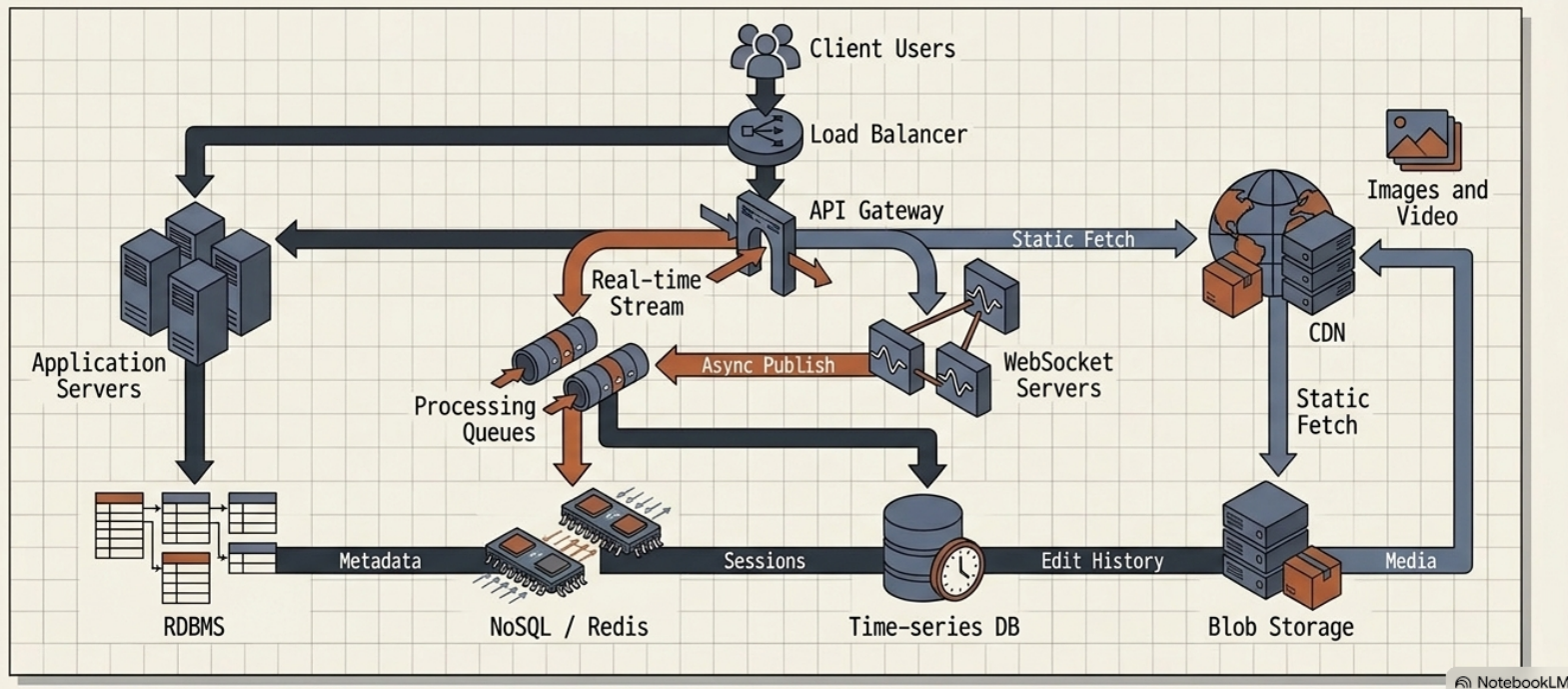
Fast retrieval for comments and active sessions.

RDBMS

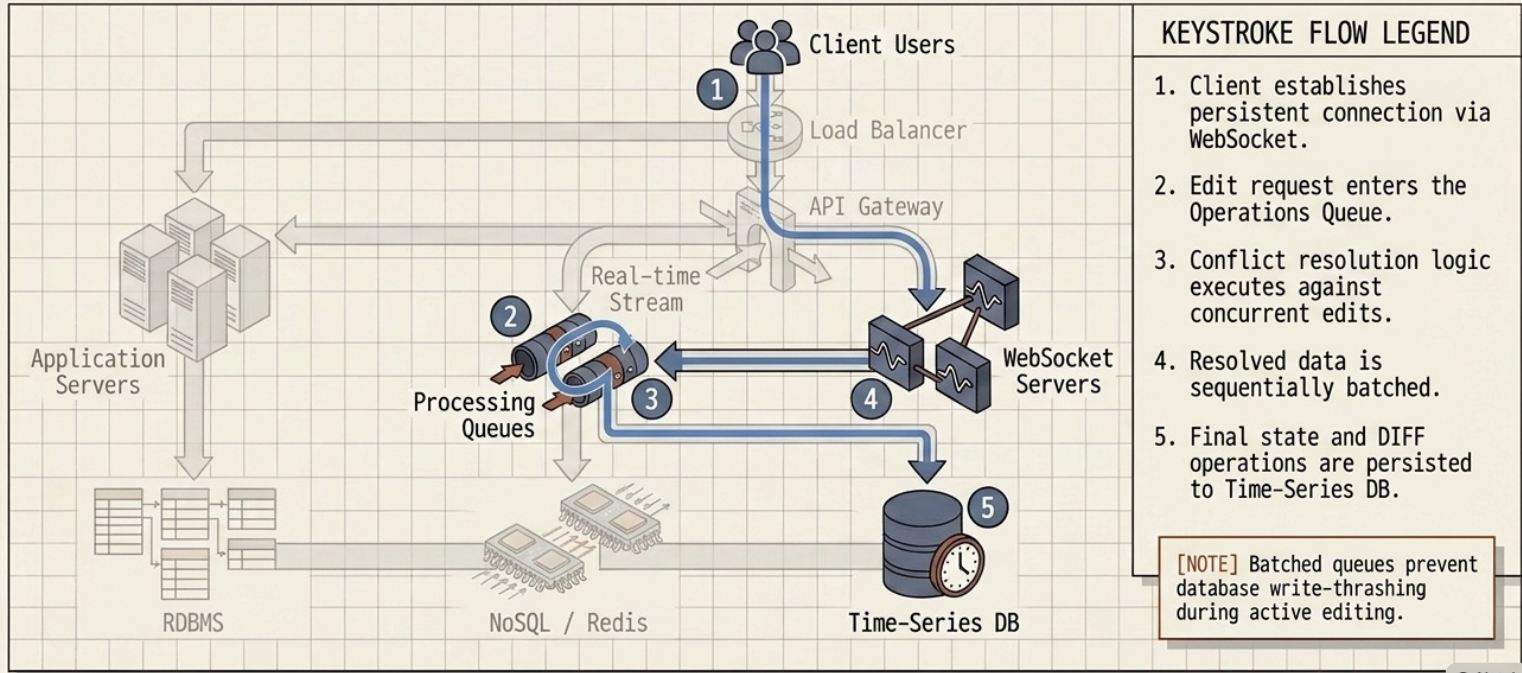


Enforces privilege restrictions and user metadata.

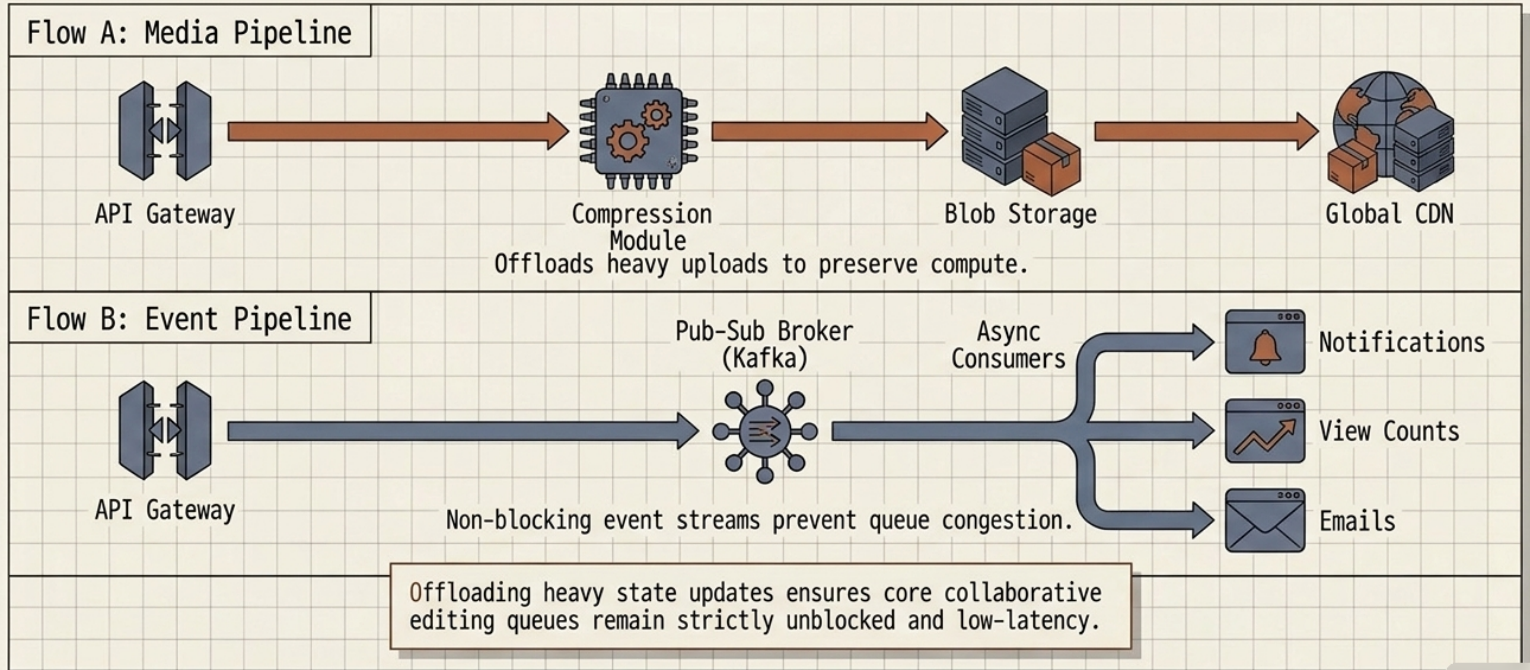
The macro-architecture separates real-time operations from static assets



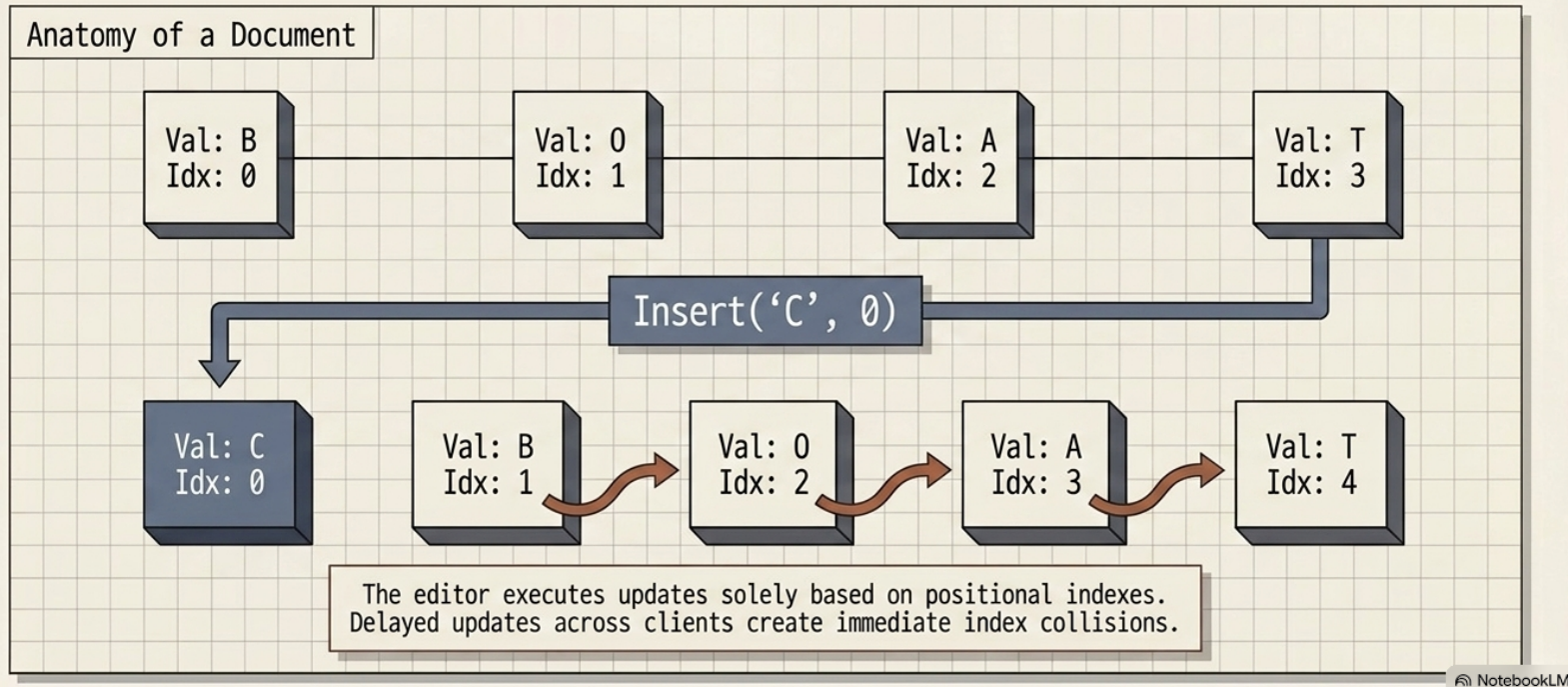
Tracing a real-time keystroke through the WebSocket processing queue



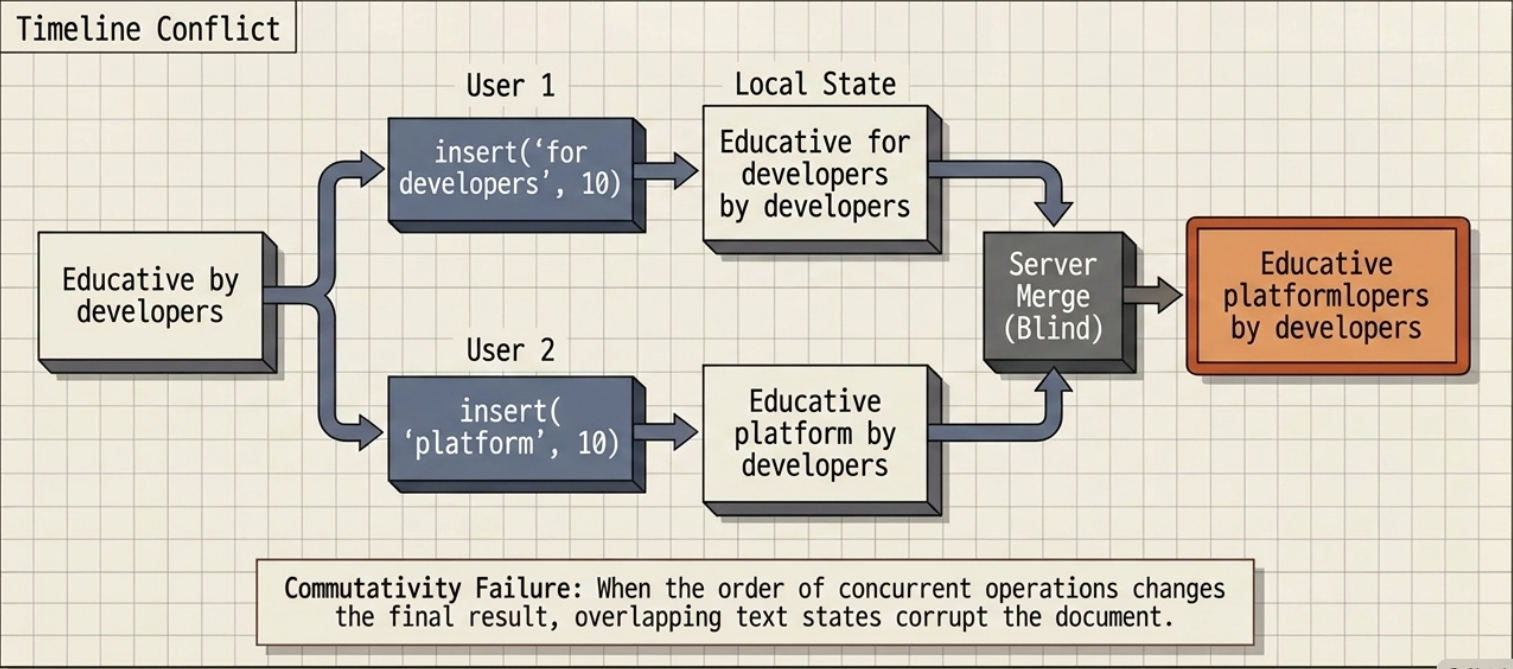
Asynchronous components handle non-critical workflows without blocking edits



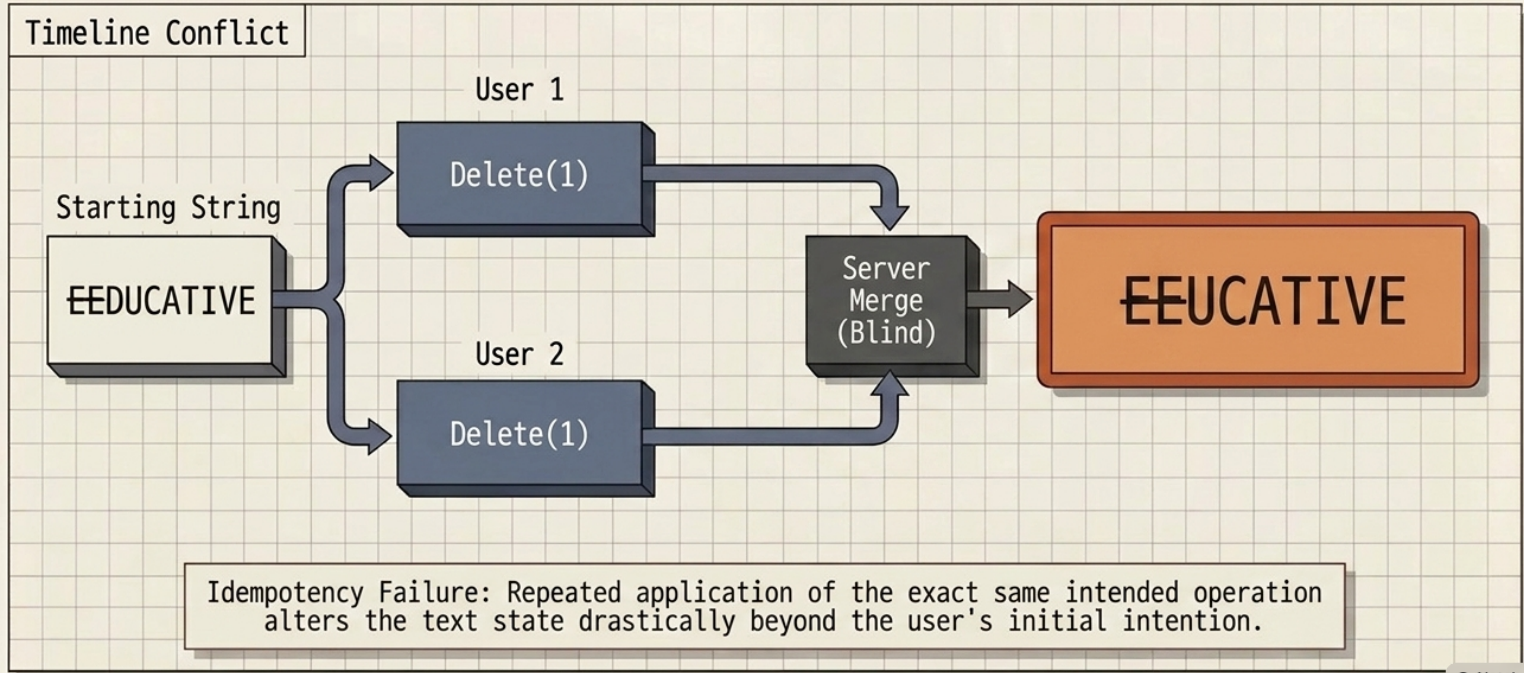
A document is an array of objects defined by value and positional index



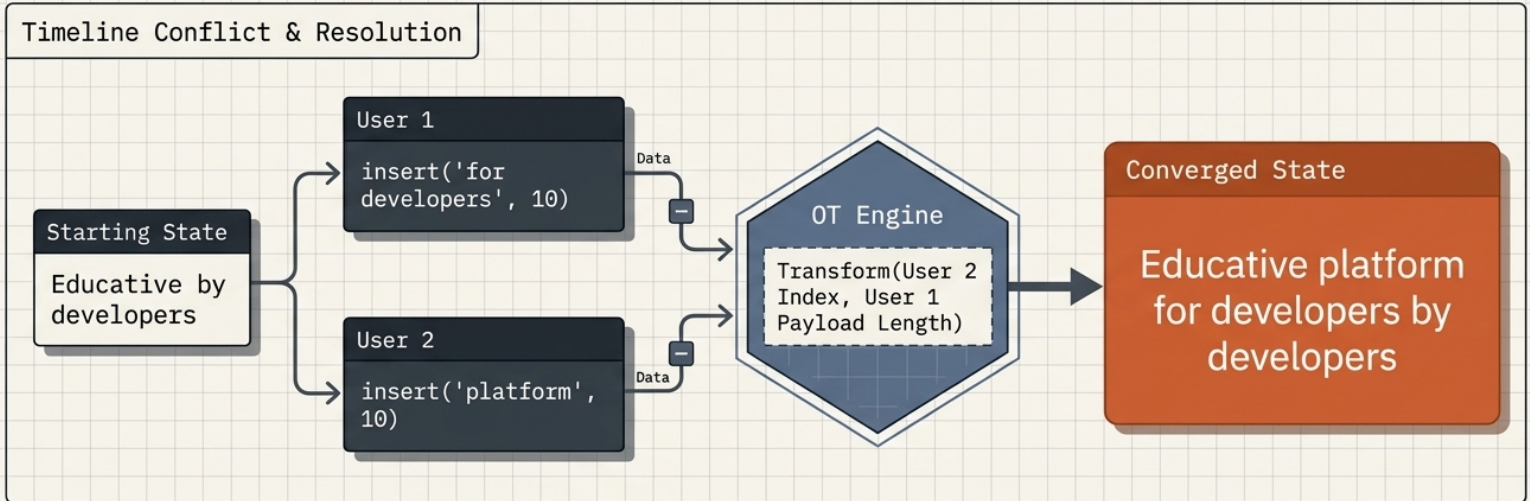
Unresolved concurrent insertions destroy state commutativity



Unresolved concurrent deletions violate operation idempotency

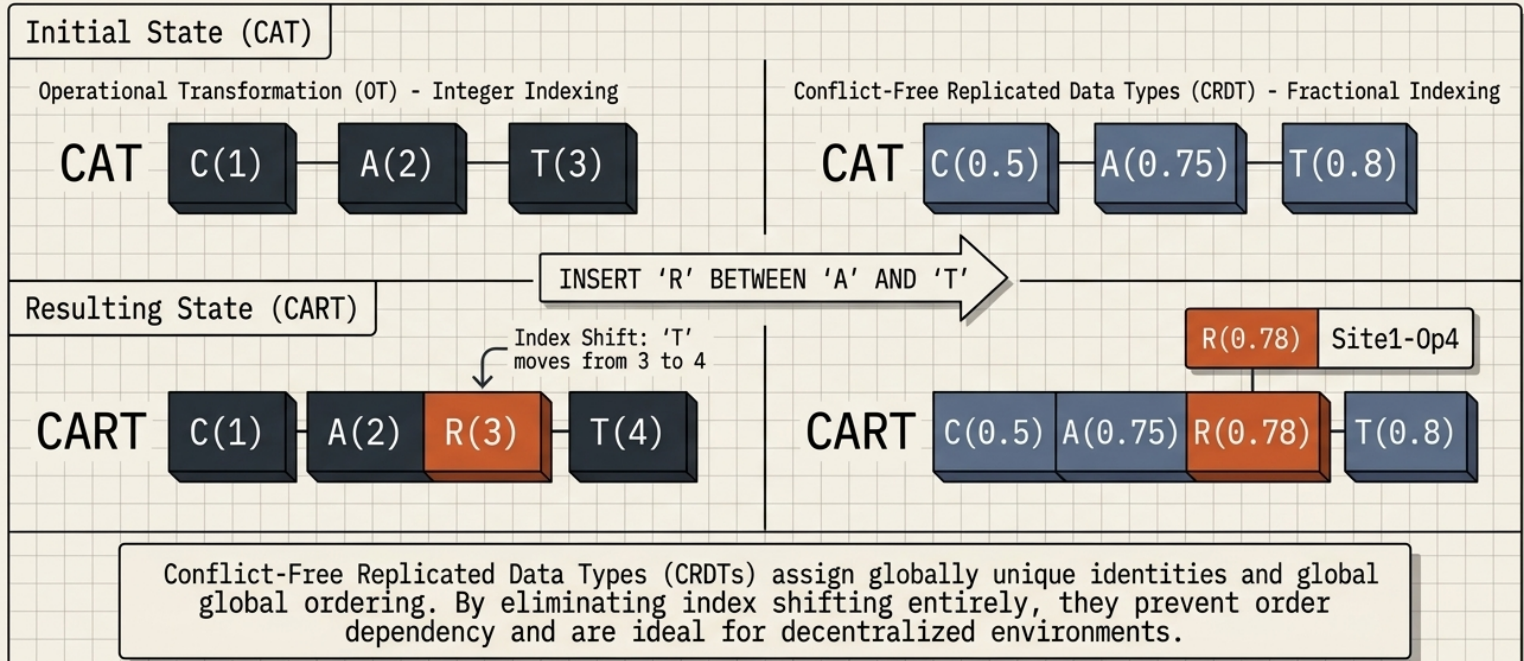


Operational Transformation (OT) maintains consistency via index shifting



OT is a lock-free, non-blocking algorithm that dynamically transforms positional indexes. It enforces the CC consistency model (Convergence of document replicas across all clients).

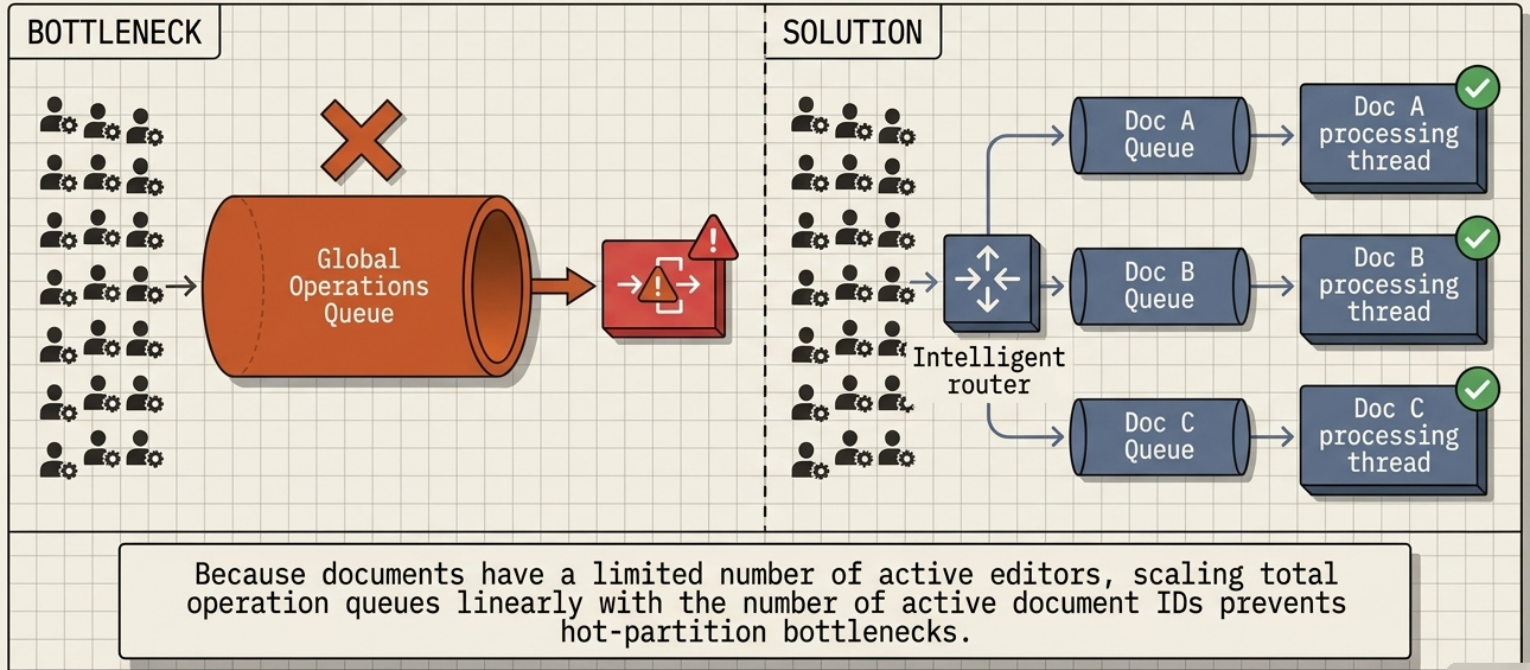
CRDTs simplify resolution via fractional indexing and global uniqueness



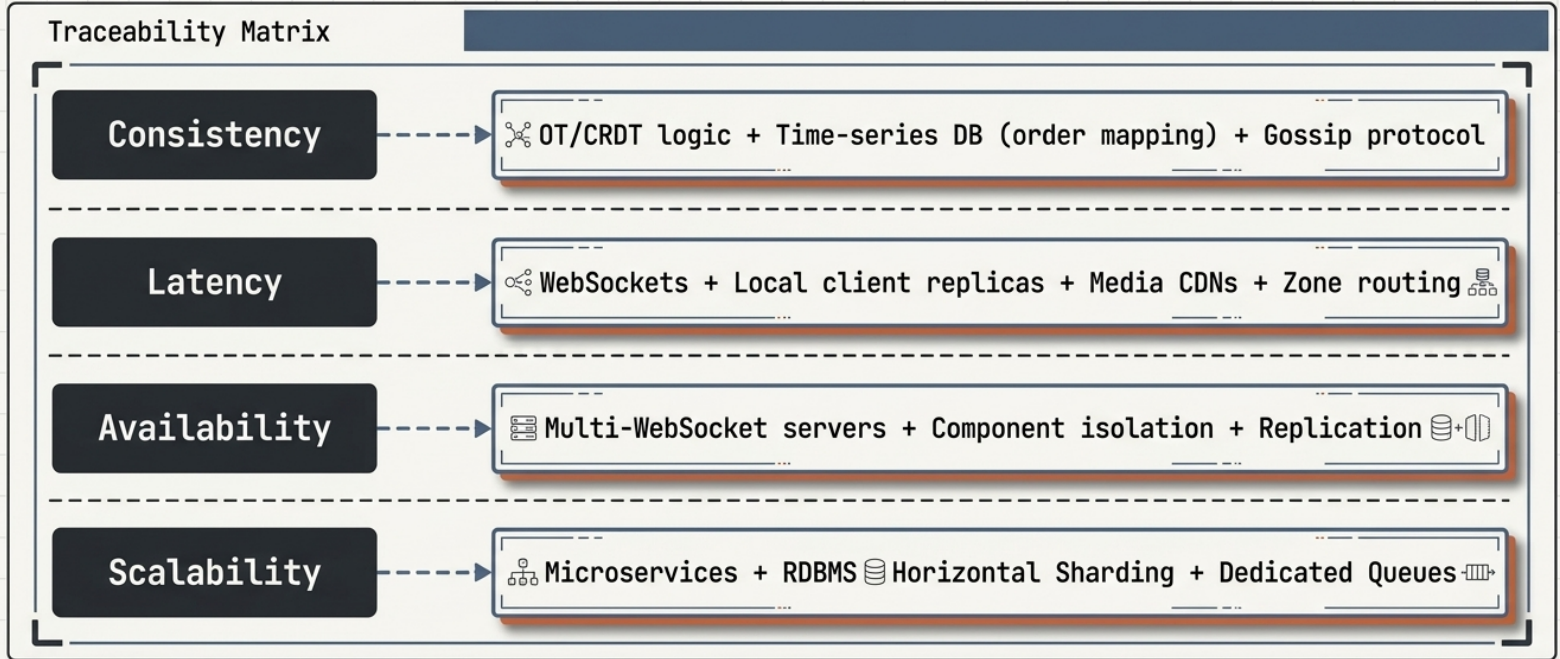
Diagnostic Matrix: Operational Transformation vs. CRDT

DIMENSION	OPERATIONAL TRANSFORMATION (OT)	CRDT (CONFLICT-FREE REPLICATED DATA TYPES)
Data Structure	Integer positional index	Fractional index + Global Unique ID
Order Dependency	Highly dependent; shifts required	Strictly order independent
Complexity	Notoriously complex server logic	Complex data structure, simpler logic
Consistency	Strong consistency / CC Model	Eventual consistency
Primary Use Case	Centralized client-server (Google Docs)	Serverless, P2P, modern collaboration

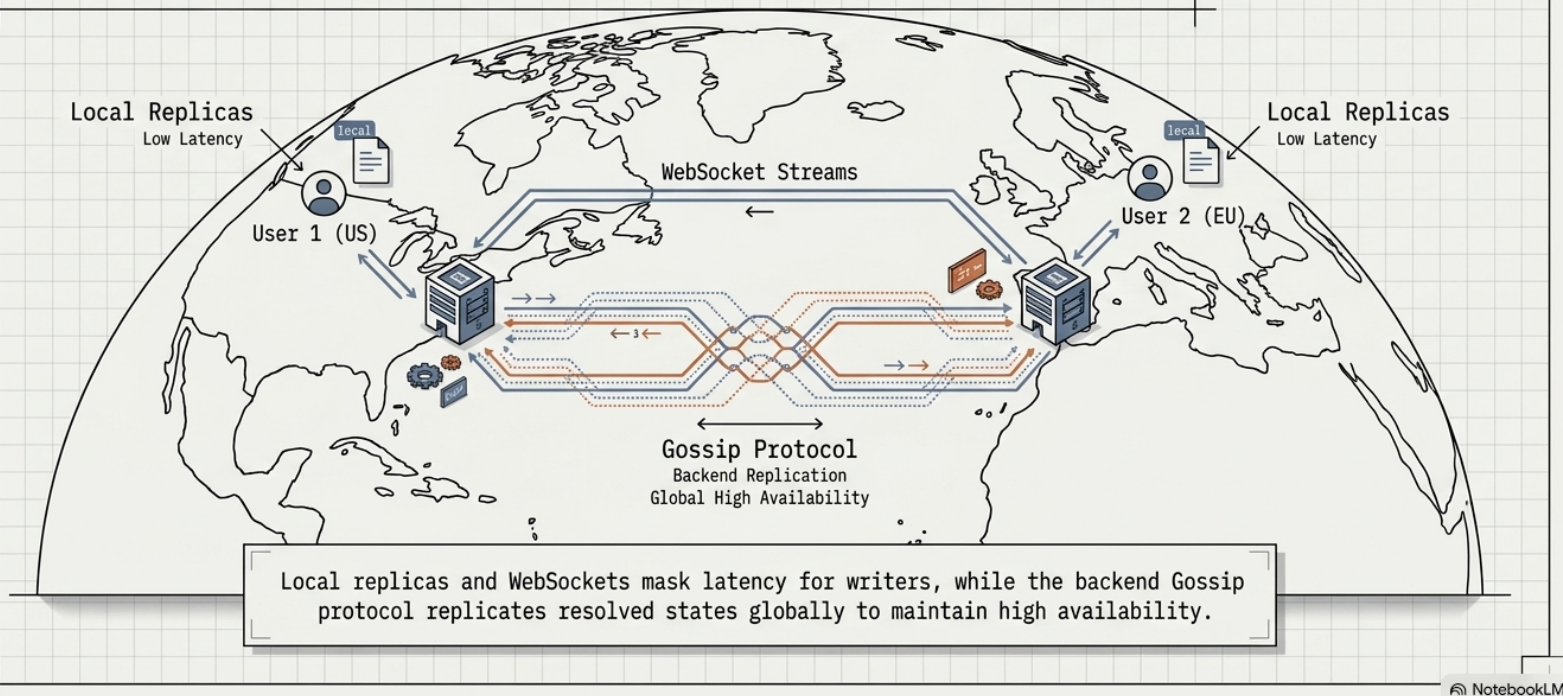
Scaling horizontally requires dynamically assigning queues per active document



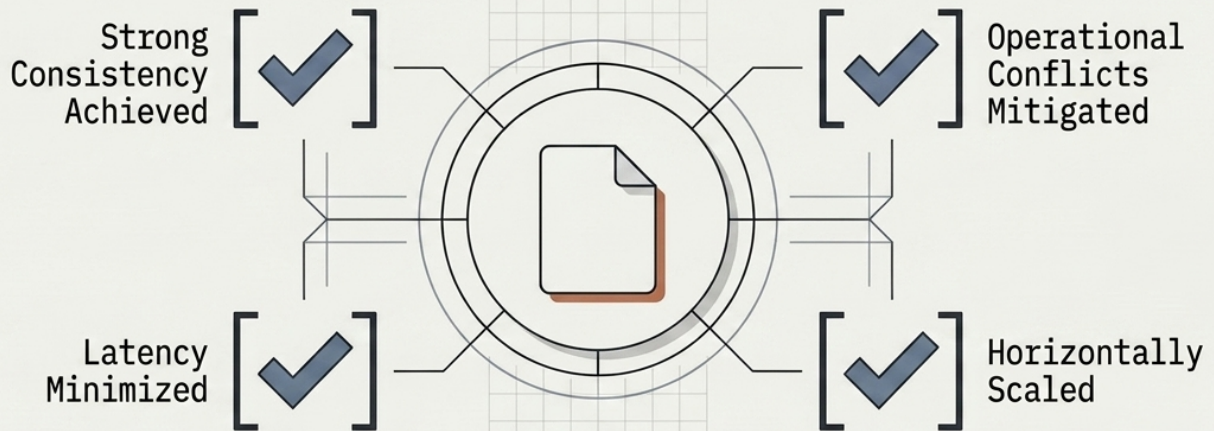
The system architecture maps directly to the initial non-functional constraints



Ensuring strong consistency across geographically distributed nodes



Achieving continuous convergence at global scale



By pairing persistent WebSocket transport with strict Operational Transformation, the architecture translates chaotic concurrent user inputs into a singular, synchronized source of truth.